Real-Time Detection of Editing Surges on Wikipedia

ID2221 Data-Intensive Computing, HT26 · Final project proposal · Curteis Yang, Group 18

1. Problem description

Wikipedia is edited continuously by humans and by bots across roughly 900 projects. A sudden concentration of edits on one page normally means one of three things: a breaking news event, a vandalism episode or edit war, or an automated bot run. Telling these apart is useful to the editor community, but only while the surge is still happening, which a nightly batch job cannot deliver.

This project investigates whether a streaming pipeline can detect per-page editing surges in near real time over an unbounded event stream and label them bot-driven or human-driven. The detection rule is kept deliberately simple; the questions the project actually investigates are the ones the course is concerned with. How does event-time windowing with watermarks behave when the revision timestamp lags stream arrival? How much operator state does a stateful aggregation accumulate over a long-tailed key space such as page titles, and how does key choice affect partition skew? What throughput and end-to-end latency does the pipeline sustain, and how do watermark and trigger interval trade latency against completeness? And does checkpointed state recover correctly after a failure?

2. Tools

Everything runs locally under Docker Compose so a grader can reproduce it without any cloud account or credential. Versions are pinned because the connectors constrain each other.

- **Apache Kafka 3.9 (KRaft)** as the durable partitioned log buffering the ingested stream. Messages are keyed by `(wiki, page_title)` rather than by wiki: three projects account for about 71% of feed volume, so keying by wiki alone would put most of the load on three partitions while still giving no stronger ordering guarantee than the per-page aggregation needs.
- **Spark Structured Streaming (PySpark 3.5.x)** for event-time windowed aggregation and surge scoring, with the RocksDB state store provider.
- **Spark SQL** in batch mode for the reference distributions computed over the archived event history.
- **MinIO** (S3-compatible object store) holding the archive as date-partitioned Parquet. This is the big-data storage system the batch path reads from, and it is what makes the streaming results reproducible.
- **Apache Cassandra 5.0** as the NoSQL serving layer, via spark-cassandra-connector 3.5.1. Windowed results are keyed `PRIMARY KEY ((wiki, day), window_start, page)` with `window_start DESC`: bucketing on day bounds partition growth, which a wiki-only partition key would not, and the dashboard reads a time slice with one partition read.
- **Streamlit** for a small dashboard reading from Cassandra. Java 17 throughout.

3. Data

The Wikimedia EventStreams *recentchange* feed (stream.wikimedia.org/v2/stream/recentchange), a public Server-Sent Events endpoint requiring no authentication. Each event is a JSON record carrying wiki, page title, namespace, user, bot flag, change type, old and new page length, and timestamps. Sampled on 22 September 2026 the feed delivers about 35 events per second, of which roughly half are automatic *categorize* records and about 16.5 per second are real page edits, so the pipeline filters to `type in ('edit','new')` and works from roughly 1.4 million edits per day.

A small Python producer subscribes to the feed and publishes to Kafka; the same records are archived to Parquet in MinIO, partitioned by date and hour. The producer projects each event down to the ten fields the pipeline uses, which is what keeps the archive tractable: the full payload averages 1.4 kB per event, or about 4.2 GB per day, while the projected record is 249 bytes and compresses to an estimated 0.15 GB per day in Parquet. Around two weeks of collection therefore gives roughly 20 million edit events in a few gigabytes. The archive is the input to the batch reference job, and it also lets the stream be replayed at a controlled rate for throughput measurement and for a grader who cannot reproduce a live feed.

4. Methodology and algorithm

1. **Ingest.** Producer writes to the Kafka topic `wiki-edits`, keyed as above, with exponential-backoff reconnection using the SSE `Last-Event-ID` header so that disconnections do not silently leave holes in the archive.
2. **Reference distributions.** Spark SQL over the Parquet archive computes the empirical distribution of per-page per-minute edit counts, and per (wiki, namespace, hour of day in UTC) the typical edit rate and bot share. A mean-plus-k-sigma threshold is deliberately not used for the page-level signal: the sampled per-page count distribution is zero-inflated and heavy-tailed (93% of active pages receive a single edit per sampling interval), so a high empirical quantile is the appropriate reference and sigma is not meaningful.
3. **Streaming aggregation.** Structured Streaming reads Kafka, parses the JSON, and uses the MediaWiki revision timestamp as event time. That field was measured to lag stream arrival by a median of 1.7 s and up to 67 s, which sets the watermark; the job groups by (wiki, page, one-minute tumbling window) and computes edit count, distinct editors, bot fraction and net bytes changed.
4. **Surge scoring.** A window is flagged when its edit count exceeds the archive-derived high quantile for that namespace and involves at least a minimum number of distinct editors, the second condition separating genuine contention from one bot iterating. Flagged windows are labelled bot-dominated or human-dominated by bot fraction. The rule is intentionally simple and explainable; the contribution is the pipeline, not the detector.
5. **Serving.** A `foreachBatch` sink upserts aggregates and alerts into Cassandra, and the dashboard shows a live top-N of surging pages together with a per-wiki edit-rate timeline.
6. **Evaluation.** Measured on replay from the archive: sustained throughput against the number of Kafka partitions and Spark cores, and against the two candidate partition keys, to quantify the skew effect; end-to-end latency from revision timestamp to visibility in Cassandra across trigger intervals; the effect of watermark length on state size, completeness and latency; and a fault-injection run that kills the driver mid-stream and checks that checkpointed offsets and state leave the Cassandra tables in the same final state as an uninterrupted run, which holds because the writes are idempotent upserts on the window key.